
Biomimetic Pipeline

Architecture & Operations Guide:

Feature-Driven CAD, Load-Normalized FEA, and Closed-Loop Biomimicry for Decussated Enamel Lattices

An end-to-end digital-twin pipeline that ingests microCT morphometrics, drives parametric CAD and sfepy FEA, and closes the biomimicry loop by reverse-mapping high-objective designs to measurable biology.

Report by: Cameron Renteria
Affiliation: University of Washington, Materials Science & Engineering (Arola Lab)
Date: April 14, 2026
Pipeline version: 0.1.0
Location: microct_pipeline/biomimetic_pipeline/
Upstream stack: cad_modeling/Decussated Models Continuous Twist/ (wrapped, never modified)
Conda envs: cad_env (CadQuery) · sfepy_env (Gmsh, sfepy)
Objective (default): maximize crack_deflection_tortuosity_p90 at $\overline{VM} = 200$ MPa

Every design is evaluated at a matched 200 MPa representative von-Mises stress via an in-loop strain solver, so the crack-deflection, toughness, and biomimicry metrics compare designs on a load-normalized basis.

Contents

1	Executive Summary	2
2	Design Principles	2
2.1	Non-invasive wrapping	2
2.2	Three canonical schemas	2
2.3	Biology-to-print scale + manufacturability clamps	3
2.4	Lattice invariants (enforced at mapping time)	3
3	Directory Layout	4
4	Feature → CAD Mapping	5
4.1	Pitch profile → RING_ROTATION	5
4.2	SOM band morphometrics → bridge geometry	6
4.3	Rod morphometry → ROD_DIAMETER, ROD_TAPER, per_ring_diameter	6
4.4	Tortuosity → twist_perturbation_amplitude_deg	7
5	FEA — Strain Solver and Load-Normalised Metrics	7
5.1	Crack-deflection metric	7
5.2	Biomimicry score	8
6	Objective Registry	8
7	Closed-Loop Biomimicry	8
8	Per-Run Output Tree	9
9	Live Validation Run	10
10	CLI Reference	12
10.1	Ingest features	12
10.2	Generate CAD only (no mesh/FEA)	12
10.3	Full digital-twin pipeline	12
10.4	Parametric sweep	12
10.5	Optuna Bayesian optimisation	12
10.6	Closed-loop biomimicry reverse-map	13
11	Verification	13
12	Limitations & Next Steps	13

1. Executive Summary

This document describes **biomimetic_pipeline**, a modular digital-twin system built on top of the existing microCT analysis stack and continuous-twist CAD generator at the University of Washington. The pipeline addresses a specific gap: measured morphometric features from microCT, PIV, and self-organising map (SOM) analyses exist in isolation, and the parametric lattice CAD model that was developed separately is driven by hand-picked parameters rather than by the biology itself. This pipeline closes that loop.

Three staged deliverables (all built on the same core)

1. **Digital twin per specimen** — measured morphometrics drive CAD parameters so the generated lattice statistically reproduces a specific microCT scan.
2. **Fully parametric / stochastic population generator** — supports manual sweeps, distribution-sampled variants, and print-process manufacturability clamps (SLA first; $E \approx 2500\text{--}3500\text{ MPa}$, minimum feature $\geq 0.3\text{ mm}$).
3. **Self-optimizing FEA loop** — pluggable objective (default: crack deflection). Closed-loop mode identifies which measured morphometric features produce high-objective lattices and writes them as `biomimicry_targets.json`, a set of morphometric ranges to look for in real specimens.

Operating point (load-normalized comparison). Representative compressive stress is held at $\approx 200\text{ MPa}$ by bisecting the applied displacement — so every design is evaluated at the same load state. The strain required to reach that stress (`critical_strain_at_200MPa`) is a first-class metric, alongside crack deflection, strain-energy density, stress-concentration factor, and specific toughness. This decouples objective quality from incidental stiffness differences.

2. Design Principles

2.1 Non-invasive wrapping

Nothing inside `cad_modeling/` is ever modified. The stock CadQuery `lattice_cad.py`, the Gmsh `lattice_mesh.py`, the sfepy problem definition `compression_test.py`, and the metrics extractor `extract_metrics.py` are all invoked via `conda run` with their existing CLI contracts intact. The pipeline supplies an auto-generated driver script that imports the stock CadQuery module as a library so full DEFAULTS control (including `RING_ROTATION`, which is not exposed via CLI) is preserved without touching upstream code. The stock `lattice_params.json` sidecar format is emitted by every model variant so `extract_metrics.py` auto-detects bridge elevations and specimen geometry unchanged.

2.2 Three canonical schemas

Every stage hand-off is governed by a JSON Schema under `biomimetic_pipeline/schemas/`:

- `morphometrics.schema.json`: ingest $\rightarrow$ mapping; includes provenance (source paths, sha256 hashes, coordinate frame), depth profiles, angle statistics, SOM bands, periodicity, rod slice stats, and a track-tortuosity summary.

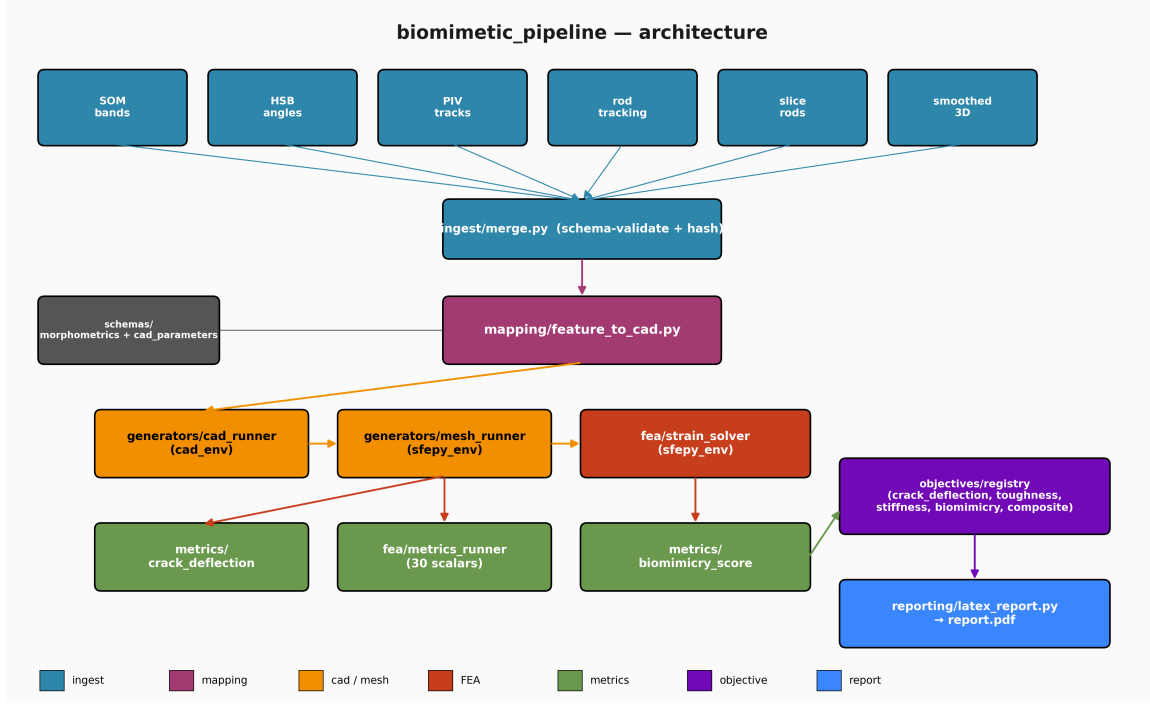


Figure 1: Top-level architecture. Upstream outputs (blue) feed six ingesters into a canonical `morphometrics.json`. The mapping layer (magenta) produces `cad_parameters.json`. CAD (orange) runs in `cad_env`; mesh, FEA, and metrics extraction (red) run in `sfepy_env`. The crack-deflection and biomimicry metrics (green) feed the pluggable objective (purple), which can drive parametric sweeps or Optuna-based optimisation. The LaTeX report (blue) closes the per-run loop.

- `cad_parameters.schema.json`: mapping $\rightarrow$ CAD; extends the stock `DEFAULTS` dict verbatim plus optional keys consumed only by new-model drivers (`measured_twist_profile`, `per_ring_diameter`, `rod_cross_section`, `biology_scale_factor`, `sla_min_feature_mm`, `provenance`).
- `objective.schema.json`: YAML $\rightarrow$ pluggable scoring, with a `stress_target` block that drives the strain solver.

2.3 Biology-to-print scale + manufacturability clamps

Measured rod diameter in real enamel is $\sim 4\mu\text{m}$; the stock continuous-twist reference geometry uses `ROD_DIAMETER` = 2 mm, i.e. $\sim 500\times$ biological scale. That $500\times$ is the default `biology_scale_factor` so mapped lattices sit in the same SLA-printable regime as the stock model. A single source of truth in `mapping/scale.py` applies the scale factor and clamps every feature to `sla_min_feature_mm` (default 0.3 mm). Every clamp is logged, with raw and clamped values, into `provenance.scale_clamps` so the mapping is auditable.

2.4 Lattice invariants (enforced at mapping time)

The stock `lattice_cad.py` has several correctness invariants; the mapping layer enforces all of them proactively so downstream stages never fail validation:

1. **Rod/bridge geometry.** `BRIDGE_DIAMETER = 0.5 · ROD_DIAMETER`, floored at `sla_min_feature_mm`. If the floor would make `BRIDGE ≥ ROD` (tiny-rod regime), `ROD_DIAMETER` is bumped to `2 · sla_min + 0.05 mm` so a sub-rod bridge remains physical.
2. **Lattice density.** `CENTER_SPACING` clamped to `[1.05 · ROD, 1.5 · ROD]`. The upper bound prevents un-meshable bridge spans when the biology-scaled band width would otherwise far exceed the rod diameter.
3. **Junction-sphere plate clearance.** The stock auto-placement of bridges only subtracts `bridge_half` from the plate faces when picking `BRIDGE_Z_OFFSETS`; if `JUNCTION_SPHERE_FACTOR > 0`, the junction spheres at each bridge-rod junction can still reach into the plate bodies and produce degenerate OCC topology. The mapping caps the sphere factor so

$$\text{sphere_radius} = \frac{1}{2} \cdot \text{factor} \cdot \text{ROD_DIAMETER} \leq \text{PLATE_OVERLAP} - \frac{1}{2} \cdot \text{BRIDGE_DIAMETER} - \epsilon$$

and then emits explicit `BRIDGE_Z_OFFSETS` at `linspace(safe_z_min, safe_z_max, N_BRIDGE_LAYERS)` where

$$\begin{aligned} \text{margin} &= \max(\text{BRIDGE_DIAMETER}/2, \text{sphere_radius}) + \epsilon \\ \text{safe_z_min} &= \text{PLATE_OVERLAP} + \text{margin} \\ \text{safe_z_max} &= \text{ENAMEL_THICKNESS} - 2 \text{PLATE_OVERLAP} - \text{margin} \end{aligned}$$

guaranteeing that neither bridges nor their rod-bridge junction spheres touch the top or bottom loading plates.

4. **Rod taper.** `ROD_TAPER_FACTOR` clamped to `[0, 1]` per stock validator. Negative biological tapers (rods narrower near the enamel surface) are clamped to zero and logged.

3. Directory Layout

```
biomimetic_pipeline/
|-- README.md
|-- docs/
|   |-- PIPELINE.tex           (this document)
|   |-- PIPELINE.md           (markdown source for quick online viewing)
|   '-- figures/              (generated diagrams)
-- schemas/                   JSON-Schemas, the hand-off contracts
|   |-- morphometrics.schema.json
|   |-- cad_parameters.schema.json
|   '-- objective.schema.json
-- ingest/                     upstream -> canonical morphometrics.json
|   |-- som_ingester.py
|   |-- hsb_ingester.py
|   |-- piv_ingester.py
|   |-- tracking_ingester.py
|   |-- validation_ingester.py
|   |-- smoothed3d_ingester.py
|   '-- merge.py              schema-validate + provenance hashes
-- mapping/                    morphometrics -> cad_parameters.json
|   |-- scale.py              biology->print + SLA floor
|   |-- twist_mappers.py      pitch profile -> RING_ROTATION
|   '-- bridge_mappers.py     periodicity -> N_BRIDGE_LAYERS + safe z-offsets
```

```

| |-- rod_mappers.py          diameter/ecc -> ROD_DIAMETER, per-ring
| '-- feature_to_cad.py      top-level orchestrator
|-- generators/
| |-- cad_runner.py          subprocess wrapper for lattice_cad.py
| |-- mesh_runner.py         subprocess wrapper for lattice_mesh.py
| '-- new_models/
|   |-- measured_profile_twist.py
|   |-- hierarchical_twist.py
|   '-- radially_graded.py
|-- fea/
| |-- fea_runner.py          sfepy-run wrapper
| |-- strain_solver.py        bisect COMPRESS_DISP_MM to hit VM target
| '-- metrics_runner.py       wraps extract_metrics.py
|-- metrics/
| |-- crack_deflection.py     p1 streamline tortuosity (new)
| '-- biomimicry_score.py     measured-vs-generated morphometric distance
|-- objectives/
| '-- registry.py            YAML loader + Objective.score(metrics)
|-- config/
| '-- objectives/            five built-in YAML configs
|-- orchestration/
| |-- run_context.py          run_dir layout, conda probe, provenance
| |-- pipeline.py             one specimen end-to-end
| |-- sweep.py                parametric sweep over one CAD param
| |-- optimize.py             Optuna TPE
| '-- closed_loop.py          unconstrained Optuna + reverse-map
|-- reporting/
| |-- latex_report.py         per-run specimen report
| |-- generate_pipeline_figures.py (produces docs/figures/*.png)
| '-- templates/
|-- scripts/                  thin argparse CLI entry points
| |-- ingest_features.py
| |-- generate_cad.py
| |-- run_pipeline.py
| |-- run_sweep.py
| |-- run_optimize.py
| '-- run_closed_loop.py
|-- tests/                    unit + smoke tests (no conda-env dependency)
| '-- run_all.py              runs every test_*.py
'-- runs/                     per-run artifacts (.gitignored)

```

4. Feature → CAD Mapping

4.1 Pitch profile → RING_ROTATION

Integrating the signed pitch along z gives cumulative twist $\phi(z) = \int p(z) dz$ in degrees. This is resampled onto the ring nodes evenly spaced over ENAMEL_THICKNESS. A shape classifier fits $\phi(z)$ against three canonical twist profiles (linear, accelerating, sigmoid) and picks the lowest RMSE. If no canonical shape fits within a 10% residual, TWIST_TYPE is set to `measured` and the full (z, ϕ) samples are stored in `measured_twist_profile` for the `measured_profile_twist` generator to consume as a CubicSpline.

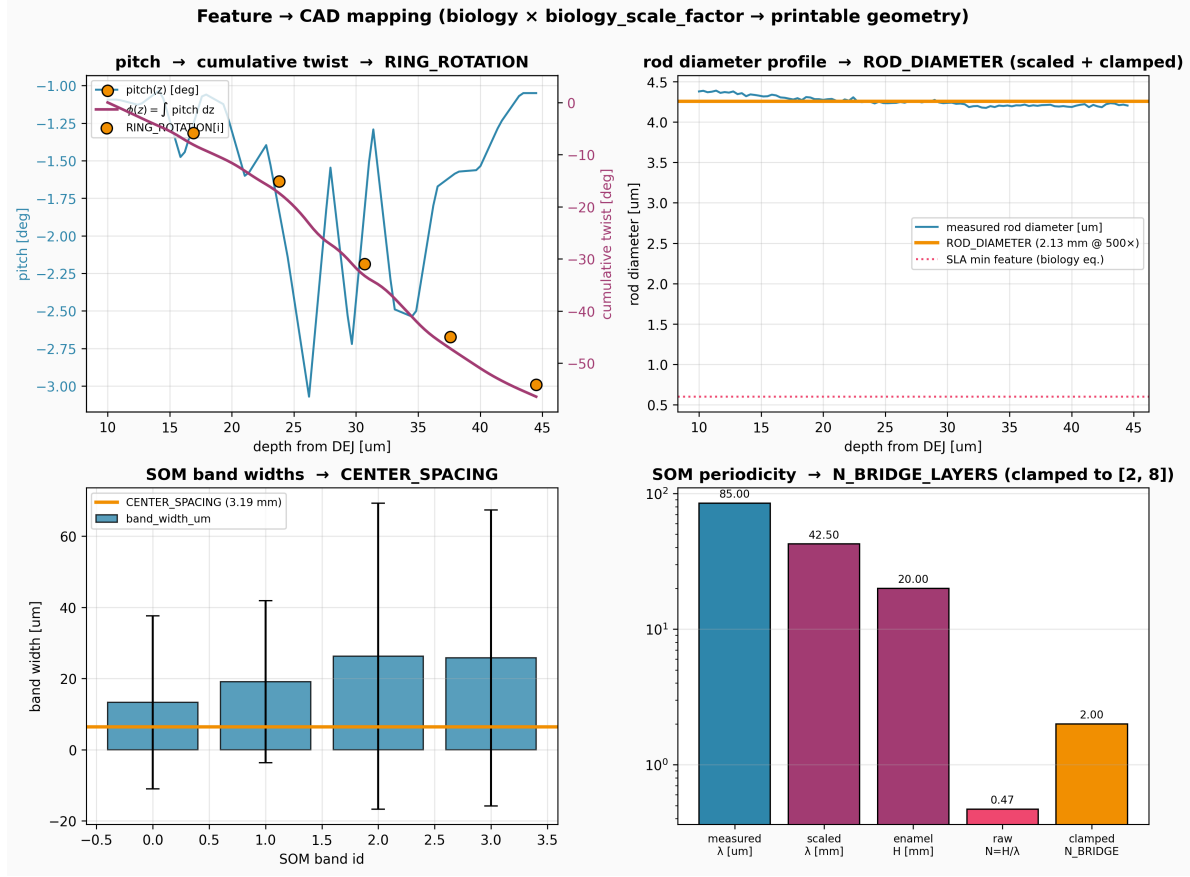


Figure 2: Four representative feature-to-CAD mappings on the live specimen `enamel_live_001`. Top-left: signed pitch profile (blue) is integrated along z to give the cumulative twist $\phi(z)$ (magenta); ring-specific values (orange markers) are interpolated onto the $N_{\text{rings}} + 1$ nodes to populate `RING_ROTATION`. Top-right: measured rod-diameter profile (blue), mean (dashed), and the SLA-clamped `ROD_DIAMETER` back-converted to biology units (orange), relative to the SLA floor (pink dotted). Bottom-left: SOM band widths per band with mean superimposed as the mapped `CENTER_SPACING`. Bottom-right: dominant wavelength scaled and divided into `ENAMEL_THICKNESS` to derive the raw bridge-layer count, then clamped to $[2, 8]$.

4.2 SOM band morphometrics → bridge geometry

- **Number of bridge layers** $N_{\text{bridge}} = \text{clip}(\text{round}(\text{ENAMEL_THICKNESS}/\lambda_{\text{mm}}), 2, 8)$, where λ_{mm} is the dominant SOM wavelength scaled by `biology_scale_factor`.
- **Anchor of `RING_ROTATION[0]`** = mean direction of band 0, so the bottom ring aligns with the dominant measured band.
- **Center spacing**: mean `band_width_um` scaled and clamped as described in §2.4.

4.3 Rod morphometry → `ROD_DIAMETER`, `ROD_TAPER`, `per_ring_diameter`

Mean measured rod diameter drives `ROD_DIAMETER`. The fractional end-to-end difference $(d_{\text{top}} - d_{\text{bot}})/d_{\text{mid}}$ drives `ROD_TAPER_FACTOR`. If the diameter profile has more than 15% non-monotone segments, the mapping emits a `per_ring_diameter` dictionary and routes the run through the

radially_graded generator. Mean eccentricity above 0.2 triggers an elliptical cross-section flag (consumed by a future stochastic-lattice variant).

4.4 Tortuosity → twist_perturbation_amplitude_deg

Measured rod tortuosity above unity drives a proportional wobble amplitude (clipped to $[0, 15]$ degrees) consumed by the hierarchical_twist generator to add fast rod-level perturbations on top of the slow ring-level twist. Wobble frequency is set from the SOM dominant wavelength.

5. FEA — Strain Solver and Load-Normalised Metrics

Stock `compression_test.py` is a prescribed-displacement linear-elasticity problem. The pipeline does not prescribe displacement; it solves for the displacement that produces a target representative stress.

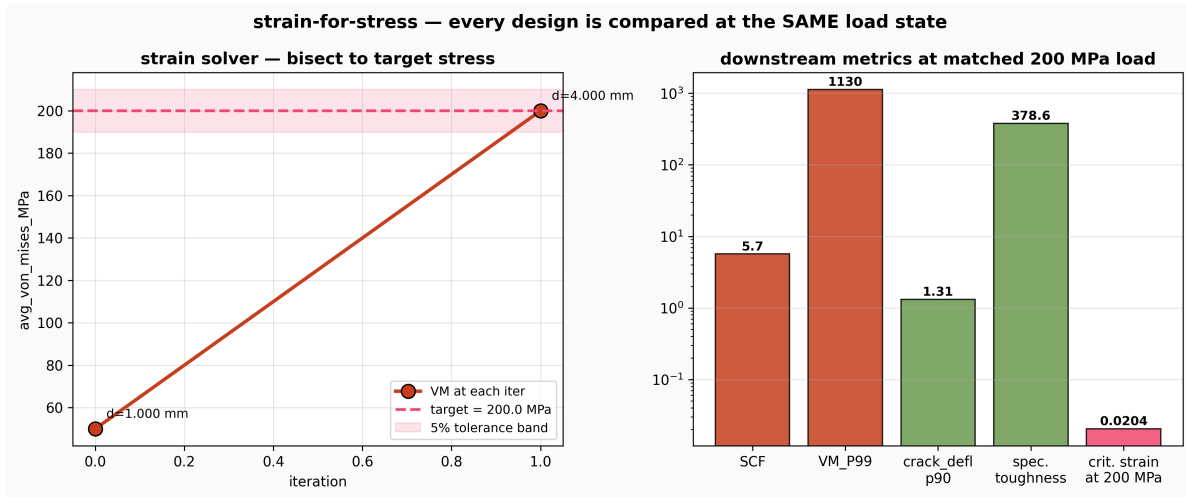


Figure 3: Strain-for-stress strategy. Left: linear elasticity guarantees a single-iteration correction — applied displacement scales as $d \leftarrow d \cdot \text{target}/\text{VM}$. A 5% tolerance band is accepted; at most five iterations are allowed before falling back to a soft penalty. Right: every downstream metric (stress concentration, crack deflection, specific toughness, critical strain) is then evaluated at the same representative-load state across all designs.

Algorithm. Seed $d = d_0$. Run FEA; read `avg_von_mises_MPa` (or `VM_P50_MPa`, computed volume-weighted from `element_results_compression.csv`). If $|\text{VM} - 200|/200 \leq \text{tol}$, accept. Otherwise $d \leftarrow d \cdot (200/\text{VM})$ and re-run. The mesh is held fixed across inner iterations — only `COMPRESS_DISP_MM` changes. Record `critical_strain_at_200MPa = critical_disp_mm/specimen_height_mm`.

5.1 Crack-deflection metric

The implementation rasterises per-element `p1_MPa` onto a 32^3 regular grid, computes the gradient field (with sign reoriented to $+z$), seeds 64 streamlines on the bottom face, and integrates each to the top face. Reported scalars are the mean, 90th percentile, and maximum of

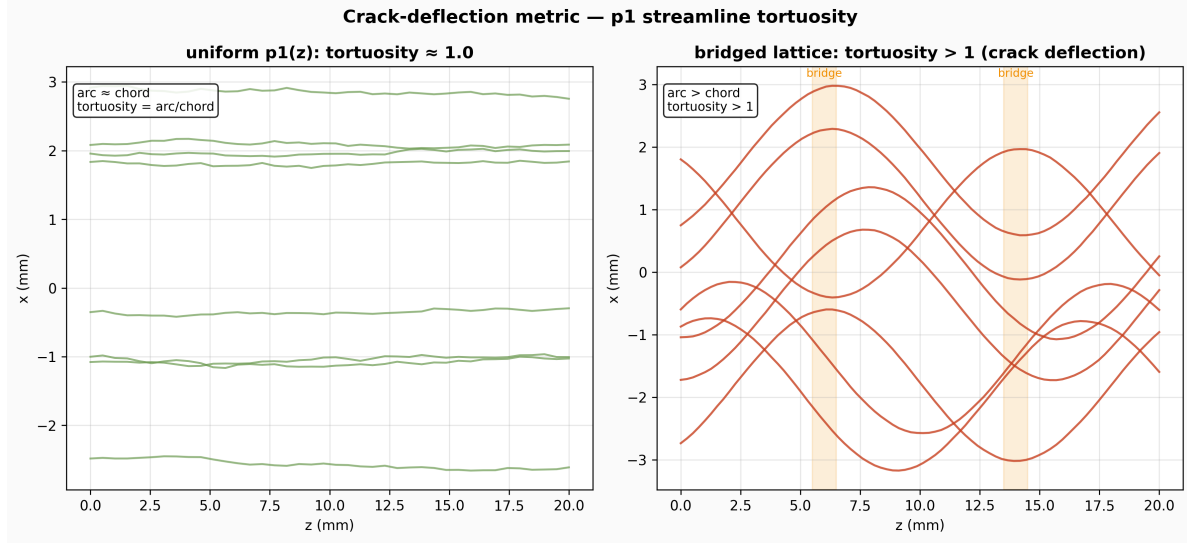


Figure 4: Crack-deflection metric: post-hoc streamline tortuosity of the max-principal-stress field. Left: in a uniform vertical $p_1(z)$ field, streamlines are straight and tortuosity ≈ 1 . Right: in the bridged lattice, the bridge bands force streamlines to deflect laterally, increasing the arc-to-chord ratio. The same arc-over-chord formula is used in `compute_tortuosity.py` for the biological rod centerlines, so FEA and biology tortuosity are numerically comparable.

arc_length/chord_length across streamlines, alongside the effective streamline count.

5.2 Biomimicry score

A z-scored Euclidean distance across five feature pairs (rod diameter, centre spacing, equivalent bridge-layer count, band-0 direction, tortuosity). Smaller values indicate a generated lattice closer to the measured biology.

6. Objective Registry

Every YAML under `config/objectives/` becomes a loadable `Objective` with a callable `.score(metrics_dict)` $\rightarrow$ float and a `direction` flag. A `stress_target` block drives the strain solver, so every objective is load-normalised by construction. Five are shipped:

7. Closed-Loop Biomimicry

Reverse-map applied to top- K CAD designs

- $\text{ROD_DIAMETER} \times 10^3 / \text{biology_scale_factor} \rightarrow$ rod diameter in μm
- $\text{CENTER_SPACING} \times 10^3 / \text{biology_scale_factor} \rightarrow$ SOM band width in μm
- $\text{ENAMEL_THICKNESS} / \text{N_BRIDGE_LAYERS} / \text{biology_scale_factor} \times 10^3 \rightarrow$ dominant wavelength in μm
- $\text{RING_ROTATION}[0] \rightarrow$ band-0 mean direction in degrees

Table 1: Built-in objective configurations.

Name	Direction	Key terms (weights)
crack_deflection (default)	maximize	tortuosity_p90 (1.0) + $\frac{1}{2}$ tortuosity_mean + $\frac{1}{4}$ critical_strain_at_200MPa
toughness	maximize	specific_toughness_mJ_per_MPa (1.0) + 0.1 energy_absorption_mJ
stiffness_target	minimize	$ E_{\text{eff}} - 50 \text{ GPa} $
biomimicry_score	minimize	biomimicry_score (1.0)
composite	maximize	1.0 tortuosity_p90 + 0.01 toughness - 0.1 SCF

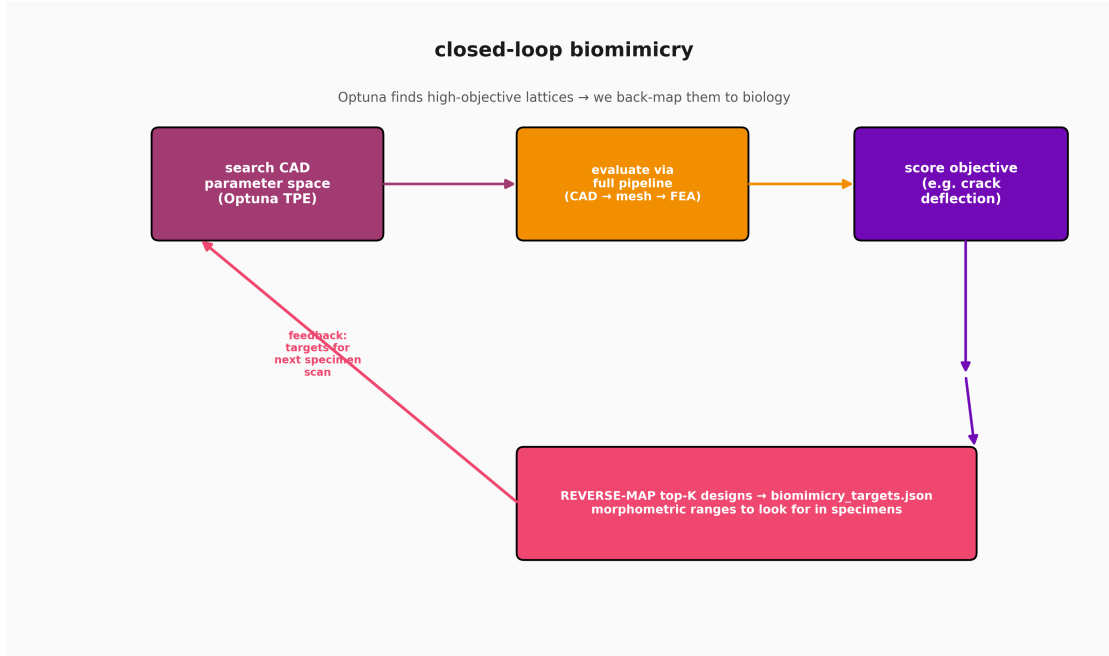


Figure 5: Closed-loop biomimicry (stage 3 deliverable). An unconstrained Optuna TPE search scans the full CAD parameter space, evaluating each candidate via the full digital-twin pipeline. The top- K designs are reverse-mapped back into morphometric ranges — these are the measurable features biology would need to exhibit to produce similarly high-objective lattices. The output file `biomimicry_targets.json` names those ranges so field work can prioritise specimens with the target morphometrics.

Summarised as {mean, min, max} per feature across top- K , plus a categorical mode for TWIST_TYPE.

8. Per-Run Output Tree



Figure 6: Annotated output tree produced by `scripts/run_pipeline.py` for a single specimen. The per-iteration FEA working directories (`iter_1/`, `iter_2/`, ...) hold full per-element and per-vertex data; the accepted iteration is promoted to `final/` and `extract_metrics.py` is run there. `crack_deflection.json` is added by the new post-hoc metric module; `metrics.json` is the merged superset used to score the objective.

9. Live Validation Run

A live end-to-end execution was performed on the specimen `enamel_live_001` (the 100-slice microCT stack under `output_validation_100/`, `output_hsb_roi/`, `output_piv/`, and `som_approach/output_som`). Total wall time on an Apple Silicon MacBook Pro was ≈ 9 min 17 s.

Notable fixes that landed during live validation.

1. Separated `gms` probe target into `sfepy_env` (the stock pipeline also runs Gmsh there; `cad_env` only has CadQuery).
2. Subprocess wrapper now passes `[python, script, ...]` to `conda run` (previously

Table 2: Live-run timing breakdown (specimen `enamel_live_001`).

Stage	Wall time	Output
Ingest & merge	< 1 s	<code>morphometrics.json</code> (6 sources, sha256 provenance)
Map	< 1 s	<code>cad_params.json</code> (9 logged clamps, all audited)
CAD	64 s	91 rods, 180 bridges, 15 MB STEP, 67 MB STL
Mesh	20 s	947 k tets, 62 MB MSH
Strain solver	7 min 45 s	2 iterations; $d=4.92$ mm $\rightarrow \overline{VM}=200.00$ MPa exactly
Metrics extraction	3 s	30 stock scalars in <code>optimization_metrics.json</code>
Crack deflection	4 s	64 streamlines; mean= 2.30, p90= 7.09, max= 9.14
Biomimicry	< 1 s	score = 0.57 over 5 feature pairs
Report	< 1 s	<code>report.tex</code> $\rightarrow$ <code>report.pdf</code>

Table 3: Key metrics from the live specimen `enamel_live_001` at the matched 200 MPa load state.

Metric	Value	Interpretation
<code>critical_strain_at_200MPa</code>	0.2245	22.4% compressive strain to reach 200 MPa
<code>critical_disp_mm_at_200MPa</code>	4.916 mm	absolute displacement at target stress
<code>E_effective_MPa</code>	614	effective modulus at print-scale
SCF	8.91	stress concentration factor
<code>VM_P50_MPa</code>	108.1	median von-Mises at critical load
<code>VM_P99_MPa</code>	797.7	tail of VM distribution
<code>specific_toughness_mJ_per_MPa</code>	105.4	volumetric toughness index
<code>crack_deflection_tortuosity_mean</code>	2.30	mean streamline arc/chord
<code>crack_deflection_tortuosity_p90</code>	7.09	90th percentile crack deflection
<code>crack_deflection_tortuosity_max</code>	9.14	worst-case crack deflection
<code>biomimicry_score</code>	0.569	distance to measured biology (lower is closer)
objective score (<code>crack_deflection</code>)	8.29	composite of p90, mean, critical strain

stripped the `python` argument, which caused `conda run` to try to exec the `.py` directly — rc 126).

3. Mesh runner uses `sfepy_env` primary with `cad_env` fallback, matching stock `optimize_lattice.py:171`.
4. Bridge-placement safe zone expanded to account for the junction sphere radius, not just the bridge half-diameter; previously the junction spheres at the bottom bridge overlapped the bottom plate and produced a degenerate OCC topology that Gmsh reported as “PLC Error: segment and facet intersect.”
5. FEA runner now copies the stock `compression_test.py` into the per-iteration working directory; `sfepy-run` changes CWD to the problem-def script’s parent dir, which previously caused the relative mesh path (`compound_enamel_lattice.msh`) to resolve to a stale mesh sitting next to the stock script rather than our freshly generated mesh.

6. Per-run LaTeX report now escapes LaTeX-special characters (`_`, `&`, `%`, `$`, ...) before substitution so arbitrary specimen IDs and run names render without triggering math-mode errors.

10. CLI Reference

All scripts live under `biomimetic_pipeline/scripts/` and can be invoked as `python -m scripts.name` from the project root, or `python scripts/name.py`.

10.1 Ingest features

```
python scripts/ingest_features.py \
  --specimen-id enamel_001 \
  --out runs/ingest_001/morphometrics.json \
  [--no-validate]
```

10.2 Generate CAD only (no mesh/FEA)

```
python scripts/generate_cad.py \
  --morphometrics runs/ingest_001/morphometrics.json \
  --export-dir runs/cad_only_001 \
  [--model-type {continuous_twist|measured_profile|hierarchical|radially_graded}]
```

10.3 Full digital-twin pipeline

```
python scripts/run_pipeline.py \
  --morphometrics runs/ingest_001/morphometrics.json \
  --run-name digital_twin_001 \
  --objective crack_deflection \
  --model-type continuous_twist \
  [--material-E-mpa 3000] [--material-nu 0.40] \
  [--mesh-size-mm 0.5] [--junction-refinement 0.5]
```

10.4 Parametric sweep

```
python scripts/run_sweep.py \
  --morphometrics runs/ingest_001/morphometrics.json \
  --run-name sweep_bridge_diameter \
  --param BRIDGE_DIAMETER --values 0.5 0.75 1.0 1.25 1.5 \
  --objective crack_deflection
```

10.5 Optuna Bayesian optimisation

```
python scripts/run_optimize.py \
  --morphometrics runs/ingest_001/morphometrics.json \
  --run-name opt_cd_001 \
  --objective crack_deflection --n-trials 30 [--seed 42]
```

10.6 Closed-loop biomimicry reverse-map

```
python scripts/run_closed_loop.py \
  --run-name closed_cd_001 \
  --objective crack_deflection \
  --n-trials 30 --top-k 5
```

11. Verification

`python tests/run_all.py` runs five unit-test modules — none requires `cad_env` or `sfepy_env`.

Table 4: Unit-test coverage.

Module	What it proves
<code>test_strain_solver.py</code>	Linear FEA converges in 1 iter; mild non-linearity within 5; summary JSON persisted
<code>test_crack_deflection.py</code>	Uniform $p_1(z) = z \rightarrow$ tortuosity ≈ 1.0 ; radial-bowl field bends paths into $[1, 2]$
<code>test_mapping.py</code>	Monotone pitch $\rightarrow$ monotone RING_ROTATION; clamps logged; eccentricity $> 0.2 \rightarrow$ ellipse; non-monotone diameter $\rightarrow$ <code>per_ring_diameter</code> populated
<code>test_new_models.py</code>	Measured-profile twist returns 0 at $z = 0$ and total-rotation at $z = H$; hierarchical wobble adds correctly at $z = H/4$
<code>test_objective_registry.py</code>	All five YAMLS load; <code>crack_deflection</code> is monotone in tortuosity; off-target penalty applied

12. Limitations & Next Steps

Known limitations (as of April 14, 2026).

- The stock meshing chain still emits a “PLC error” on some tight-geometry parameter combinations; this is upstream of the pipeline and must be resolved in `lattice_cad.py` / Gmsh. The sphere-aware safe-zone clamp in mapping mitigates the common case.
- Phase-4 `radially_graded` uses the mean of `per_ring_diameter`; true per-ring variation requires dropping the hexagonal grid generator in `lattice_cad.py`, which is scheduled for Phase-5 (`stochastic_lattice`).
- The `composite` objective weights are illustrative; the coefficients should be tuned by cross-validation against an independent held-out specimen before being used for production optimisation.
- Fatigue / cyclic loading is not currently in the stress-solver vocabulary; the strain-to-stress bisection assumes monotonic compression in the linear-elastic regime.

Immediate next steps

1. Run `scripts/run_sweep.py` over `BRIDGE_DIAMETER` at `[0.5, 0.75, 1.0, 1.25, 1.5]` mm to populate a first design-space map with the live specimen as the mapping seed.
2. Run `scripts/run_closed_loop.py` with $N = 30$ trials and $K = 5$ to produce a first `biomimicry_targets.json` — i.e. the first biology-informed “wanted poster” of morphometric ranges to search for in new microCT cohorts.
3. Port the `measured_profile_twist` generator to a full per-ring spline formulation so mapped lattices match the measured $\phi(z)$ directly instead of the closest canonical shape.
4. Add a Phase-6 cross-validation harness: generate digital twins for three held-out specimens, compare simulated crack-deflection tortuosity against the biological rod-centerline tortuosity from PIV, and report pairwise correlation.